CSC343H5 Lecture 10

Indexes and Query Optimization

Not Naaz Sibia (naaz.sibia@utoronto.ca)

Admin

- Midterm grades out!
  - Remark request form on Piazza
  - We will go over the midterm next week!
- Exam review lectures starting next week
- My office hours are back to being in-person today (11-12) and online on Friday (12-1)

Activity

MONEY :D

I need two volunteers!

What you just built

Server A	Server B
SELECT → 1	
	SELECT → 1
seats > 0 ✓	
	seats > 0 ✓
UPDATE → 0	
	UPDATE → 0
INSERT ticket	
	INSERT ticket

Two tickets sold. One seat exists.

The read went stale

Server B read remaining BEFORE Server A wrote it. B's decision was based on a value that stopped being true.

You cannot fix this in the app

Faster code makes the window smaller, not zero. An if-check just re-reads the same stale value. The fix has to live **below your code**.

What you just built

Server A	Server B
SELECT → 1	
	SELECT → 1
seats > 0 ✓	
	seats > 0 ✓
UPDATE → 0	
	UPDATE → 0
INSERT ticket	
	INSERT ticket

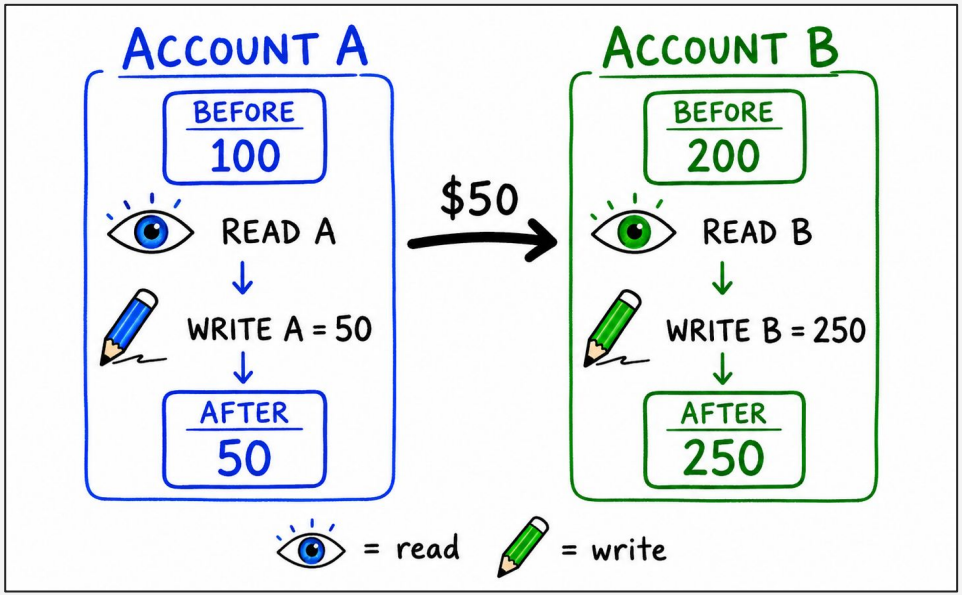
Two tickets sold. One seat exists.

The read went stale

Server B read remaining **BEFORE** Server A wrote it. B's decision was based on a value that stopped being true.

A Transaction

A database's view of updates - a sequence of **reads** and **writes**.



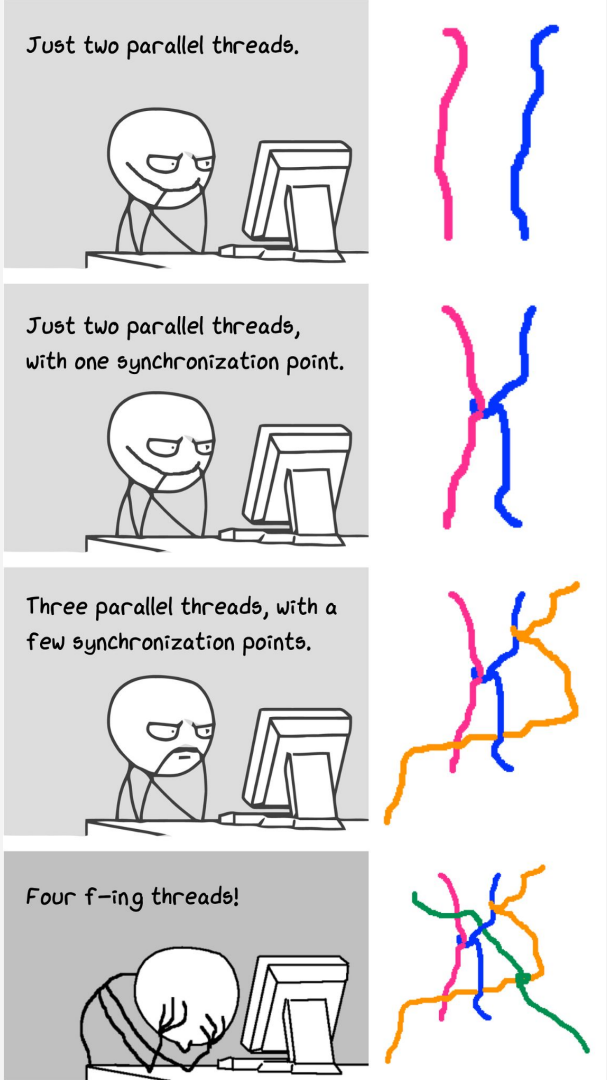
Concurrency

What is Concurrent Process (CP)?

- Multiple users access databases and use computer systems simultaneously.

Example: Airline reservation system

- An airline reservation system is used by hundreds of travel agents and reservation clerks concurrently.
- Banking system: you may be updating your account balances the same time the bank is crediting you interest.



Concurrency

Why Concurrent Process?

- Better transaction throughput and response time
- Better utilization of resource

Concurrency

1. Interleaved Processing
   - Concurrent execution of processes is interleaved in a single CPU.
2. Parallel Processing
   - Processes are concurrently executed in multiple CPUs.

ACID: four promises, one per failure mode

A **Atomicity**

All the actions happen, or none do. No half-sold tickets after a crash mid-transaction.

C **Consistency**

The DB goes from one valid state to another valid state.

I **Isolation**

Your transaction runs as if it were alone, even when it is not. This is the hard one.

D **Durability**

Once it commits, it survives a power cut. **Committed means committed.**

The notation

`R1(A)`	Transaction 1 reads object A 👀
`W2(B)`	Transaction 2 writes object B ✏️
`C1 / A1`	Transaction 1 commits ✅ / aborts ❌
`S1(A) / X1(A)`	T1 takes a shared (read) 🔒👀 / exclusive (write) lock on A 🔒✏️
`U1(A)`	T1 releases its lock on A 🔓

There are three ways to break it!

A schedule is just the interleaving

A schedule of T1…Tn is an ordering of all their operations, with one rule:

each transaction's own operations keep their original relative order. Shuffle

the decks together, do not shuffle a deck against itself.

T1	T2
R(A)	
	R(A)
W(A)	
	W(A)
Abort	
	Commit

Written on one line:

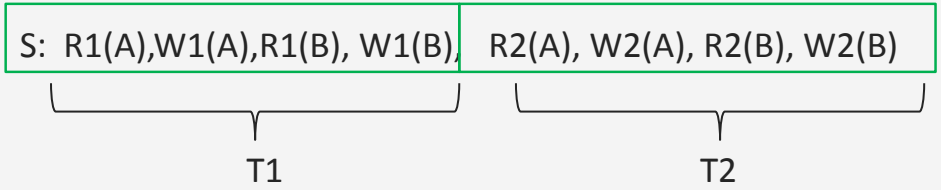
```
R1(A) R2(A) W1(A) W2(A) A1 C2
```

Scheduling Transactions

- **Serial schedule**: Schedule that does not interleave the actions of different transactions.
- **Equivalent schedules**: For any database state, the effect (on the set of objects in the database) of executing the first schedule is identical to the effect of executing the second schedule.
- **Serializable schedule**: A schedule that is equivalent to some serial execution of the transactions.

Serial Schedule

T1	T2
R(A)	
A := A+100	
R(B)	
B := B+100	
	R(A)
	A: = A* 2
	R(B)
	B:= B*2



A Serializable Schedule

T1	T2
R(A)	
A := A+100	
	R(A)
	A: = A* 2
R(B)	
B := B+100	
	R(B)
	B:= B*2

Notice: this is not a serial schedule, i.e., there is interleaving of operations

Net effect is the same as the serial schedule

S: R1(A),W1(A), R2(A), W2(A), R1(B), W1(B), R2(B), W2(B)

A Non-Serializable Schedule

T1	T2
R(A)	
A := A+100	
	R(A)
	A: = A* 2
	R(B)
	B:= B*2
R(B)	
B := B+100	

What is different?

Ordering of operations is not consistent

S: R1(A),W1(A), R2(A), W2(A), R2(B), W2(B), R1(B), W1(B)

S1 · Reading uncommitted data

T1	T2
R(A)	
W(A) A+100	
	R(A) ← reads +100
	W(A) A*1.06
	Commit
R(B)	
W(B)	
Abort	

What went wrong

T2 read a value T1 had written but not committed. T1 then aborted, so T2 built its work on a number that, officially, never existed.

Where you have seen it

A dashboard showing a payment that later vanishes. A recommendation engine trained on rolled-back orders. Silent, and very hard to reproduce.

Pattern: T1 writes → T2 reads (WR)

S1 · Reading uncommitted data

T1	T2
R(A)	
W(A) A+100	
	R(A) ← reads +100
	W(A) A*1.06
	Commit
R(B)	
W(B)	
Abort	

Dirty Read! (WR conflict)

What went wrong

T2 read a value T1 had written but not committed. T1 then aborted, so T2 built its work on a number that, officially, never existed.

Where you have seen it

A dashboard showing a payment that later vanishes. A recommendation engine trained on rolled-back orders. Silent, and very hard to reproduce.

Pattern: T1 writes → T2 reads (WR)

S2 · The value changed under you

T1	T2
R(A) → 1	
	R(A) → 1
	W(A) A = 0
	Commit
W(A) A = 0	
Commit	

Both decremented a counter of 1. It should be −1 worth of stock sold; it reads 0. One sale evaporated.

What went wrong

T1 read A, then T2 changed A while T1 was still running. If T1 read A again it would get a different answer, inside a single transaction.

Why it is nastier than it looks

Any logic of the form "read, decide, write" is unsafe. That is most business logic, including TicketRush.

`Pattern:  T1 reads → T2 writes  (RW)`

S2 · The value changed under you

T1	T2
R(A) → 1	
	R(A) → 1
	W(A) A = 0
	Commit
W(A) A = 0	
Commit	

Both decremented a counter of 1. It should be −1 worth of stock sold; it reads 0. One sale evaporated.

Unrepeatable read
(RW conflict)

What went wrong

T1 read A, then T2 changed A while T1 was still running. If T1 read A again it would get a different answer, inside a single transaction.

Why it is nastier than it looks

Any logic of the form "read, decide, write" is unsafe. That is most business logic, including TicketRush.

`Pattern: T1 reads → T2 writes (RW)`

S3 · Overwriting uncommitted work

T1 sets every salary to $1000. T2 sets every salary to $2000. The rule: A and B must always match.

T1	T2
`W(A) = 1000`	
	`W(A) = 2000`
	`W(B) = 2000`
	`Commit`
`W(B) = 1000`	
`Commit`	

What went wrong

T2 overwrote A while T1 was mid-transaction. T1 then overwrote B.

S3 · Overwriting uncommitted work

T1 sets every salary to $1000. T2 sets every salary to $2000. The rule: A and B must always match.

T1	T2
`W(A) = 1000`	
	`W(A) = 2000`
	`W(B) = 2000`
	`Commit`
`W(B) = 1000`	
`Commit`	

What went wrong

T2 overwrote A while T1 was mid-transaction. T1 then overwrote B.

Lost Update (WW conflict)

All three are the same shape

Two operations conflict when all three hold:

1. **Different transactions**

   R1(A) and W1(A) never conflict, one transaction is already ordered with itself.

2. **Same object**

   W1(A) and W2(B) are irrelevant to each other. No shared object, no problem.

3. **At least one is a write**

   R1(A) and R2(A) never conflict. Readers cannot corrupt each other.

How to test serializability!

Start from the obviously-correct case

A serial schedule runs transactions one at a time, no interleaving. It is correct by definition. Nobody can interfere if nobody overlaps.

T1	T2
R(A) · A += 100	
W(A)	
R(B) · B += 100	
W(B)	
	R(A) · A *= 2
	W(A)
	R(B) · B *= 2
	W(B)

Start from the obviously-correct case

A serial schedule runs transactions one at a time, no interleaving. It is correct by definition. Nobody can interfere if nobody overlaps.

T1	T2
R(A) · A += 100	
W(A)	
R(B) · B += 100	
W(B)	
	R(A) · A *= 2
	W(A)
	R(B) · B *= 2
	W(B)

But… disk reads are slow. While T1 waits on I/O the CPU sits idle and T2 waits in line. Throughput collapses.

Nobody runs a real database this way.

So we cheat: look serial, run interleaved

A schedule is serializable if its effect on the database is identical to SOME serial order of the same transactions. It does not have to BE serial, it has to be indistinguishable from serial.

Serializable

T1	T2
R(A), W(A)	
	R(A), W(A)
R(B), W(B)	
	R(B), W(B)

Same net effect as T1 then T2.

NOT serializable

T1	T2
R(A), W(A)	
	R(A), W(A)
	R(B), W(B)
R(B), W(B)	

T1 first on A, T2 first on B. No serial order does that.

Spot the difference: only the ORDER of the conflicting pairs changed.

Conflict serializability: a testable version

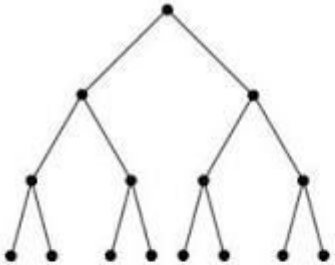
“Same effect for every possible database state” is impossible to check directly. So we use a stricter, mechanical proxy:

Conflict equivalent

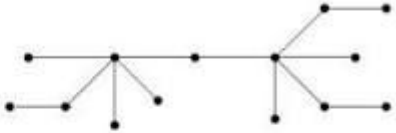
Same actions, same transactions, and every pair of conflicting actions appears in the same order.

Conflict serializable

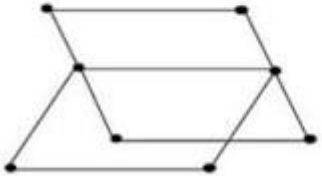
Conflict equivalent to some serial schedule. You can reach a serial order by swapping only adjacent NON-conflicting actions.



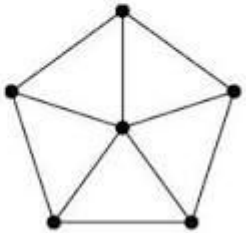
tree



lobster



book



wheel

The test: build a graph, look for a cycle

1. **One node per transaction**

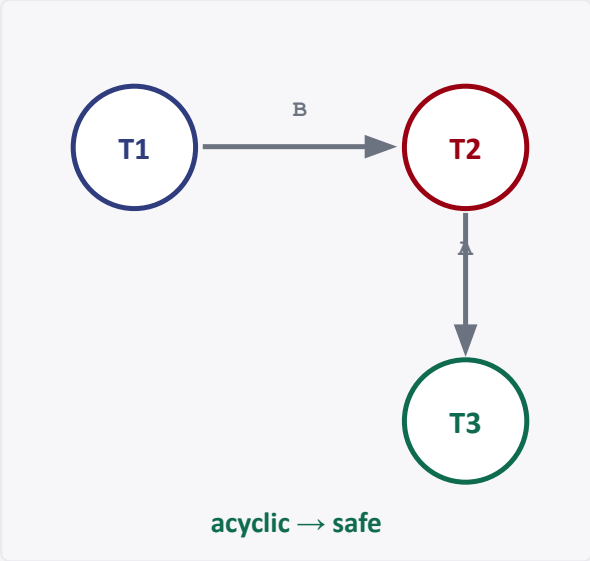
   T1, T2, T3 …

2. **Draw Ti → Tj when Ti goes first in a conflicting pair**

   Scan the schedule left to right. Every conflicting pair gives one edge, labelled with the object.

3. **Look for a cycle**

   No cycle → conflict serializable, and the topological order IS your equivalent serial schedule. Cycle → not conflict serializable.



We will need lots of practice with this!

More examples!

```
R2(A)  R1(B)  W2(A)  R3(A)  W1(B)  W3(A)  R2(B)  W2(B)
```

Acyclic: conflict serializable

`Equivalent serial order:  T1, T2, T3`

Read straight off the arrows.

More examples!

```
R2(A)  R1(B)  W2(A)  R3(A)  W1(B)  W3(A)  R2(B)  W2(B)
```

Acyclic: conflict serializable

`Equivalent serial order:  T1, T2, T3`

Read straight off the arrows.

Move one operation: now cyclic

T1 must precede T2 on one conflict, and T2 must precede T1 on another. Both cannot be true, so no serial order exists.


PROBLEM SOLVED

No…. aborts exist.

Aborts

Every schedule so far assumed everyone commits. Add one abort and a "correct" schedule can become impossible to undo.

T1	T2
R(A), W(A)	
	R(A) ← T1's value
	W(A)
	Commit
Abort	

T1 rolls back. T2 read T1's doomed value, but T2 already committed, and committed is forever. There is nothing the DBMS can do.

Cascading abort

If Tj read something Ti wrote, aborting Ti forces aborting Tj, and whatever read from Tj, and so on.

The log makes undo possible

Every write is recorded before it is applied. On crash recovery, all in-flight transactions are rolled back from the log. This is the Recovery Manager's job, atomicity and durability.

Three fences, each stricter than the last

Recoverable

A transaction may commit only after everyone it read from has committed.

Guarantees you never have to un-commit. Cascades still allowed.

Avoids Cascading Aborts (ACA)

A transaction may only read data written by committed transactions.

One abort stays one abort. ACA implies recoverable, not the reverse.

Strict

A value written by Ti is neither read NOR overwritten until Ti commits or aborts.

Undo is trivial: just restore the before-image. This is what real systems implement.

Serial ⊂ Strict ⊂ ACA ⊂ Recoverable - and conflict serializability is a separate axis crossing all of them.

Nobody checks graphs at runtime

The precedence graph is a proof technique for you, not a runtime algorithm. The DBMS cannot see the future, so it needs a rule set that PREVENTS cycles from ever forming.

A locking protocol is a set of rules each transaction must follow that makes non-serializable schedules unreachable.

1. **Every object has a lock**

   One lock per data item the scheduler tracks.

2. **Acquire before you touch**

   No read or write without first holding the lock.

3. **Blocked if taken**

   If another transaction holds an incompatible lock, you wait.

4. **Release when done**

   And exactly WHEN you release is the entire design question.

Two lock modes, one compatibility rule

S - shared (read)

Many transactions can hold S on the same object at once.

Readers do not disturb readers.

X - exclusive (write)

Only one holder, ever. Blocks every other lock, S or X.

Compatibility matrix - "can I get this lock if that one is held?"

held → / wanted ↓	None	S	X
None	OK	OK	OK
S	OK	OK	CONFLICT
X	OK	CONFLICT	CONFLICT

Notice this is exactly the conflict rule from Act 2, wearing a uniform: the only forbidden combinations are the ones where at least one side writes.

Two lock modes, one compatibility rule

S - shared (read)

Many transactions can hold S on the same object at once.

Readers do not disturb readers.

X - exclusive (write)

Only one holder, ever. Blocks every other lock, S or X.

Compatibility matrix - "can I get this lock if that one is held?"

held → / wanted ↓	None	S	X
None	OK	OK	OK
S	OK	OK	CONFLICT
X	OK	CONFLICT	CONFLICT

Notice this is exactly the conflict rule from Act 2, wearing a uniform: the only forbidden combinations are the ones where at least one side writes.

Locks alone are NOT enough

Both transactions lock correctly. Both release correctly. The result is still non-serializable.

T1	T2
L(A)	
R(A), W(A)	
U(A)	
	L(A)
	R(A), W(A)
	U(A)
	L(B), R(B), W(B), U(B)
L(B)	
R(B), W(B)	
U(B)	

On A: T1 → T2

On B: T2 → T1

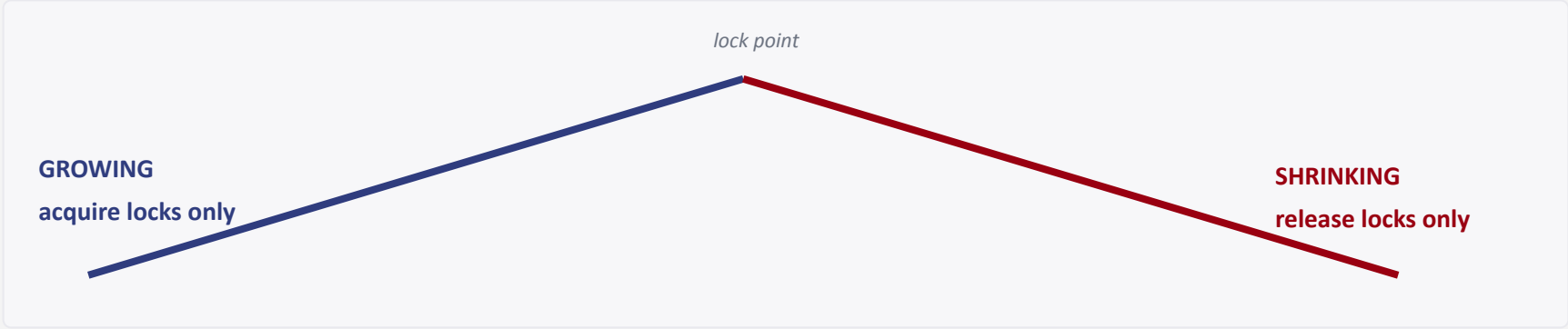
A cycle. Not conflict serializable.

The flaw

T1 let go of A and later grabbed B. In that gap T2 slipped past it on A and got ahead of it on B. Releasing early is what allows the overtaking.

Two-Phase Locking: never grab after you drop

Rule: once a transaction releases ANY lock, it may never acquire another one.



Why it works: the moment a transaction enters its shrinking phase becomes its position in the equivalent serial order. Sorting by lock point gives you a serial schedule, so no cycle can exist.

Strict 2PL: hold everything until commit

Strict 2PL keeps rule 1 and hardens rule 2: all locks are released at once, when the transaction commits or aborts.

There is no shrinking phase, just a cliff.

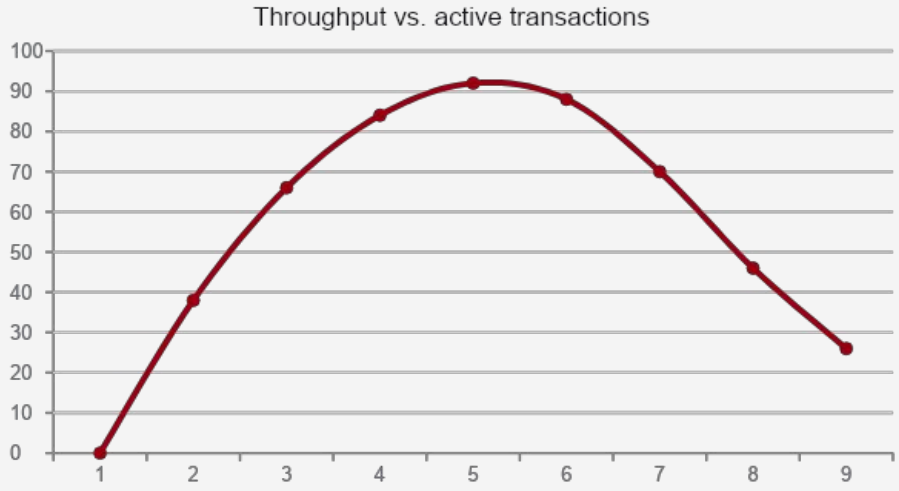
T1	T2
X(A) · R(A), W(A)	
	X(A) — DENIED, waits
X(B) · R(B), W(B)	
U(A), U(B) · Commit	
	— GRANTED —
	X(A) · R(A), W(A)
	X(B) · R(B), W(B)
	U(A), U(B) · Commit

- ✓ Conflict serializable
- ✓ Recoverable
- ✓ Avoids cascading aborts
- ✓ Strict, undo is trivial

All four guarantees for the price of one rule. This is why Strict 2PL, not plain 2PL, is the default in essentially every commercial DBMS.

Too much locking, and throughput falls off a cliff

Adding concurrent transactions helps, until blocking dominates. Past that point each new transaction mostly waits, and waits while holding locks others need. Throughput drops as load rises. That is thrashing.



Rule of thumb: if more than ~30% of transactions are blocked, reduce the number running concurrently.

Three levers you control

Lock smaller things

Row-level, not table-level. Fewer collisions, but more locks to manage. Granularity is a dial, not a setting.

Hold locks for less time

Do the slow work, API calls, file writes, user think-time — OUTSIDE the transaction. Never wrap a network call in one.

Kill your hot spots

One counter row every request touches will serialise your whole system. Shard it, batch it, or move it out of the transaction.